

Multiplayer Game Server Development

Installation

```
npm i -g typescript ts-node @types/node
npm i @slotify/gdk
```

To install private `@slotify/slotify` package you need to request private key. One of the ways to configure it is to create `.npmrc` file with the following content where `XXX` should be replaced with private key.

```
//npm.pkg.github.com/:_authToken=XXX
@slotify:registry=https://npm.pkg.github.com/
```

After configuring token you can install and start the service.

Running

```
PROVIDER=example-provider GAMES_PATH=lib/games/*/index.js node node_modules/@slotify/gdk/lib/index.js"
```

Docker

Sample `Dockerfile` (remember to replace `PROVIDER=example-provider` with name of your provider):

```
FROM node:20-slim

RUN mkdir -p /usr/src/tequity/games

WORKDIR /usr/src/tequity/games

COPY . .

RUN npm ci --omit=dev
RUN npm run build
RUN npm run obfuscate

EXPOSE 8080

CMD ["sh", "-c", "exec npm run start"]
```

Folder structure

Games should be possible to discover under path specified under `GAMES_PATH` (this path points to path after TypeScript->JavaScript compilation into `lib` directory). Default structure for `GAMES_PATH=lib/games/*/index.js` would be:

```
games/game1/index.ts
games/game2/index.ts
games/gameX/index.ts
```

Game Server

To set up a game you need to create file `index.ts` in the directory or subdirectory of path specified under `GAMES_PATH`. That file should expose object implementing `IMultiplayerGame` interface. Game needs to have its unique `name` assigned (optionally name can be an array of names - in specific cases when provider wants to run multiple identical game instances under different names).

Below example presents very simple multiplayer game.

```
export const server: IMultiplayerGame = {
  name: "example-guess-number",
  bets: {
    "main": {available: [1, 2, 5], default: 2, maxWin: 2, coin: 1},
  },
  init: async ({time, config}, random) => {
    const drawEndTime = time + 10 * 1000;
    return {state: {acceptedBets: [], drawEndTime}, nextTickTime: drawEndTime};
  },
  connected: ({time, state}) => {
    return {message: {drawEndTime: state.drawEndTime}};
  },
  command: ({time, playerId, action, bet, currency, params, config, state, betLimits}) => {
    if (
      action !== "main" || !Number.isInteger(params.luckyNumber) ||
      state.acceptedBets.some(acceptedBet => acceptedBet.playerId === playerId)
    ) {
      return {valid: false, instantTick: false, message: {betAccepted: false}};
    }
    return {valid: true, instantTick: true};
  },
  tick: async ({time, config, state, commands}, random) => {
    const wins = {};
    const cancels = [];
    const messages = {};
    const {drawEndTime, acceptedBets} = state;

    for (const {commandId, playerId, roundId, time, action, bet, currency, params} of commands) {
      if (state.acceptedBets.some(acceptedBet => acceptedBet.playerId === playerId)) {
        //important: we need to repeat this validation as only here we can guarantee current state
        cancels.push(commandId);
        messages[playerId] = {betAccepted: false};
      } else {
        state.acceptedBets.push({playerId, roundId, params, bet});
        messages[playerId] = {betAccepted: true};
      }
    }

    const drawFinished = time >= drawEndTime;
    if (drawFinished) {
      const luckyNumber = random(10);
      for (const {playerId, roundId, params, bet} of acceptedBets) {
        if (params.luckyNumber === luckyNumber) {
          const win = bet * 10;
          wins[roundId] = win;
          messages[playerId] = {isWin: true, win};
        } else {
          messages[playerId] = {isWin: false};
        }
      }
    }

    const drawEndTime = time + 10 * 1000;
    const newState = {drawEndTime, acceptedBets: []};
    return {
      state: newState,
      drawFinished,
      cancels,
      wins,
      messages,
    };
  }
};
```

```

        broadcast: {luckyNumber, drawEndTime},
        nextTickTime: drawEndTime,
    };
}

return {state, drawFinished, cancels, wins, messages, nextTickTime: state.drawEndTime};
},
simulator: {
  stats: {
    "RTP": new RTP(),
  },
  commands: (strategy, state) => {
    const playerId = "player1";
    if (!state.acceptedBets.some(acceptedBet => acceptedBet.playerId === playerId)) {
      return [
        {
          commandId: v4(),
          playerId,
          roundId: v4(),
          action: "main",
          bet: 1,
          currency: "eur",
          params: {luckyNumber: 5},
        },
      ];
    }
  },
},
};

replay() {
  throw new Error("not implemented");
},
};

export default server;

```

Bets

Bets object contains bets definition for each available *initial* commands.

Base bet action `SHOULD` be called `main`

- `available` : available bets in base currency (either array of specified items or object with `min`, `max` and `step`)
- `default` : default bet in base currency
- `maxWin` : theoretical maximum win multiplier used to calculate max exposure and max bet per operator
- `coin` : coin bet. It is only used to calculate win multiplier of base bet (`main` action). So important to keep proportion i.e. coin of `main` is 1 and coin of `bonus` 100 (if `bonus` is x100 base bet).

```

export const server: IGame = {
  bets: {
    "main": {available: [0.1, 1, 2, 5], default: 2, maxWin: 200, coin: 10},
    "bonus": {available: {min: 100, max: 200, step: 25}, default: 100, maxWin: 5, coin: 1000},
  },
};

```

Config

Optional room config to be used in simulation. Returns config for specified strategy.

```

export const server: IGame = {
  config: (strategy = "maxMultiplier1000000") => ({
    maxMultiplier: Number(input.replace("maxMultiplier", "")),
  })
};

```

```
    }},  
  };
```

Init

Init async function is triggered only once on the very beginning after the room was created. It is provided with object with the following params:

- `time` - current timestamp
- `roomId` - unique identifier of the room
- `config` - room configuration (passed from RGS, can contain room-specific settings)

Second argument is the random numbers generator:

- `random` - `(limit?: number) => number` function, see [Random Numbers Generator](#).

It is expected to return the following object:

- `nextTickTime` - any gameplay data
- `state` - (optional) player's money won

Connected

This optional function is triggered after the user got successfully connected to the game. It should be used to send current state of the game to the player so the game client can display current state; It is provided with object with the following params:

- `time` - current timestamp
- `state` - game state

It is expected to return the following object:

- `message` - message sent to the connected player

System Connected

NB! currently it is possible to use system commands through a direct connection to the websocket, not through the connector

This optional function is triggered after the system user got successfully connected to the game. It should be used to send current state of the game to the system user, so the game client can display current state; It is provided with object with the following params:

- `time` - current timestamp
- `state` - game state
- `systemId` - system id

It is expected to return the following object:

- `message` - message sent to the connected player
-

Command

This function is triggered when user sends a command. If a command maps an action from bets object it will cause withdrawal from a wallet if the command is accepted (`valid` flag).

Important: `state` can be changed in parallel so there is no guarantee that the state is the latest. The ultimate validation should happen in a tick.

It is provided with object with the following params:

- `time` - current timestamp
- `playerId` - player Id
- `action` - name of the action (if it's bet then it matches)
- `bet` - bet amount (only for betting commands)
- `currency` - bet currency (only for betting commands)
- `params` - object to pass data from the game client
- `config` - room configuration (passed from RGS, can contain room-specific settings)
- `state` - game state
- `betLimits` - bet limits object includes:
 - `minBet`: min bet in player's currency
 - `maxBet`: max bet in player's currency
 - `maxBonusBet`: max bonus bet (for non-main bets) in player's currency
 - `maxExposure`: max exposure in base in player's currency
 - `currencyRate`: currency rate to base currency (typically eur)
 - `exchangeRate`: exchange rate (from Currency feeds) to base currency (typically eur)
 - `currencyDecimals`: amount of decimal places for the currency
 - `currencyUnit`: minimal monetary value of a currency (e.g. 0.01 for "cent")
- `data?: { nickname?: string; }` - additional data passed from the rgs, contains player `nickname` from operator

It is expected to return the following object:

- `valid` - indicates if a command is valid. If it's valid it will make a bet (if it's betting command) and will be processed in the next tick
- `instantTick` - indicates if we should schedule next tick instantly
- `message` - message sent to the connected player
- `roundId` - (OPTIONAL) roundId to assign to the command - if roundId is not returned, RGS starts new round and assigns it roundId to that command

System Command

This function is triggered when system user sends a command.

Important: `state` can be changed in parallel so there is no guarantee that the state is the latest. The ultimate validation should happen in a tick.

It is provided with object with the following params:

- `time` - current timestamp
- `systemId` - system id
- `action` - name of the action
- `params` - object to pass data from the game client
- `state` - game state

It is expected to return the following object:

- `valid` - indicates if a command is valid. If it's valid it will be processed in the next tick
 - `instantTick` - indicates if we should schedule next tick instantly
 - `message` - message sent to the connected system user
-

Tick

This async function is triggered when user sends a command. If a command maps an action from bets object it will cause withdrawal from a wallet if the command is accepted (`valid` flag).

Important: `state` can be changed in parallel so there is no guarantee that the state is the latest. The ultimate validation should happen in a tick.

It is provided with object with the following params:

- `time` - current timestamp
- `config` - room configuration (passed from RGS, can contain room-specific settings)
- `state` - game state
- `drawId` - unique identifier for this draw/round
- `commands` - commands array contains
 - `commandId` - command Id
 - `playerId` - player Id
 - `roundId` - command's assigned roundId
 - `time` - time of command creation
 - `action` - name of the action (if it's bet then it matches)
 - `bet` - bet amount (only for betting commands)
 - `currency` - bet currency (only for betting commands)
 - `params` - object to pass data from the game client
- `systemCommands` - commands array contains
 - `commandId` - command Id
 - `playerId` - system Id
 - `time` - time of command creation
 - `action` - name of the action
 - `params` - object to pass data from the system user

It is expected to return the following object:

- `state` - game state - this is the only place (outside of init and cheats) to write the state
 - `drawFinished` - ...
 - `nextTickTime` - timestamp of the next tick
 - `wins` - map of player ids with win amounts
 - `cancel` - array of command ids to cancel (only player commands should be cancelled, system commands you can skip)
 - `message` - message sent to the connected player
 - `broadcast` - map of player ids with win amounts
-

Cheats

Cheats let you update the state independently of tick to be able to trigger certain scenarios.

```
export const server: IMultiplayerGame = {
  cheats: (cheat, params, state) => {
    if (cheat === "setLuckyNumber") {
      return {...state, luckyNumber: params.luckyNumber};
    }
  },
};
```

Replay

Replay transforms game state before sending the data to the game client in order to show replay in back office and game history.

```
export const server: IMultiplayerGame = {
  replay: (state, playerId) => {
    return state.cardsPerPlayer[playerId];
  },
};
```

Evaluate

Optional method is used to pass any data to promo module i.e. evaluating achievement or checking if jackpot was won.

```
export const server: IGame = {
  evaluate(type: string, draws: { state: any }[], data: any) {
    switch (type) {
      case "jackpot":
        return {jackpotWon: draws[0].state.jackpotWon};
      case "achievement":
        return {jackpotWon: draws[0].state.achivements.length > 0};
    }
  },
};
```

Regulatory - Portugal

A specific case for using `evaluate` is to provide regulatory information required in certain jurisdictions.

To ensure compliance with regulations for game launches in Portugal, it is mandatory to include `sm_result` and `descr_ap` strings that describe each game round (other "AJOG file" fields are optional). The platform can transmit these values to Portuguese operators, provided the `regulatory-pt` evaluation is implemented and the returned data adheres to the format `{ "sm_result": string, "descr_ap": string }`:

```
export const server: IGame = {
  evaluate(type: string, wagers: IWager[]) {
    switch (type) {
      case "regulatory-pt":
        return {
          "sm_result": createSmResult(wagers), // game's custom implementation
          "descr_ap": createDescrAp(wagers), // game's custom implementation
        };
        // other evaluate cases
    }
  },
};
```

Defining stats

To define what stats you want to extract you need to create `simulator` section in your game server definition.

- `stats` lists all statistics
- `config` returns initial state
- `commands` generates commands for simulator - runs before every tick

```
export const server: IMultiplierGame = {
  simulator: {
    stats: {
      "RTP": new RTP(),
      "RTP (bonus)": new RTP<ISpin[]>().filter(ticks => ticks[0].bonus),
    },
    config: (strategy) => {
      return {initialState: 123};
    },
    commands: (strategy, state) => {
      return [
        {
          commandId: "command-123",
          playerId: "player-123",
          action: "main", bet: 10,
          currency: "eur",
          params: {
            drawIndex: state.drawIndex
          }
        },
      ],
    },
  },
};
```

You have following built-in stats:

- `Average` - average value
- `AverageWin` - average win
- `Balance` - final balance
- `HitFrequency` - hit frequency 1 in ...
- `HitPercentage` - hit percentage X%
- `Iteration` - number of iterations
- `Max` - max value
- `MaxWin` - max win
- `Median` - median (warning: very slow on longer simulations)
- `MedianWin` - median win (warning: very slow on longer simulations)
- `RTP` - RTP (Return To Player)
- `TimesWin` - times win in last wager
- `Variance` - variance
- `Confidence Interval` - confidence interval for given confidence level

Each class has `filter` and `map` functions to help extracting the value you want.

Custom stats

If you need to define your custom stats you can do that by extending `Stats` class. Due to the fact simulation engine runs in multicore environment you need to implement simple map-reduce.


```

export default class CustomStats<TData = any, TState = any> extends Stats<TData, TState> {

  protected evenWins: number;

  constructor() {
    super();
  }

  value() { //returns pure value
    return this.evenWins;
  }

  message() { //returns formatted message
    return this.value()
      .toLocaleString(undefined, {maximumFractionDigits: 2})
      .replace(/,/g, " ");
  }

  protected processTicks(ticks) { //processes the wagers
    if (sumOfWins(ticks) % 2 === 0) {
      this.evenWins++;
    }
  }

  mapResults() { //send data to master core
    return {evenWins: this.evenWins};
  }

  reduceResults(result) { //received data from slave core
    this.evenWins += result.evenWins;
  }

  clearResults() { //clear
    this.evenWins = 0;
  }
}

```

Prove Fairness

This method is required for games intended to be played in **provably fair** mode. It uses cryptographic methods for random numbers generation.

This doesn't affect gameplay implementation itself, but this type of games requires proving that game outcomes deterministically depend on players' and casino's hash.

GDK handles proving that random numbers are generated from provided seeds. But game still needs to specify a sequence of randomizations and provide mapping from random numbers to "game events" specific for the game (e.g. randomising crash point multiplier in Crash game).

To achieve the above, `proveFairness` method receives randomization builder that registers what random number's `limit` a game is going to require and a `gameEventProducer` that will map the actual random number (passed by the GDK based on `limit` provided) into a game specific event: `export type IRandomizationBuilder = (limit: number, gameEventProducer: (randomNumber: number) => any) => void;`

Optionally a game can require client to send additional data (representing player choices or specific game configuration) that affects gameplay (e.g. `maxMultiplier` configured for a given room).

```

proveFairness(addRandomization: IRandomizationBuilder, data: {maxMultiplier: number}) {
  addRandomisation(2 ** 32, randomNumber => {
    const crashPointX = randomNumber;
    const crashPointMultiplier = Math.min(maxMultiplier, calculateCrashPointMultiplier(crashPointX));

    return {crashPointX, crashPointMultiplier};
  });
}

```

```
});  
}
```

Random Numbers Generator

Function `random(limit?: number): number` is used as an argument in game methods and serves as a Random Number Generator that returns values from `32-bit` range.

When `limit` is provided, the function returns integer within the range `[0, limit)` with **uniform distribution**. `limit` must be a positive integer greater than zero and not exceeding $2^{32} = 4294967296$. If no limit is specified, the function will use default maximal limit 2^{32} thus returning a random number within the full 32-bit range `[0, 232)`. An error will be thrown if the limit is not a positive integer or is greater then 2^{32} .

The underlying RNG algorithm can be selected via `--rngAlgorithm` flag of the simulation engine, or via `RNG_ALGORITHM` environment variable of the games service. There are currently two RNG algorithms available to choose from:

- `isaac` (default, certified)
- `mersenne-twister`

NOTE: When provable fairness is enabled for a game, numbers are generated via cryptographic methods using player's and casino's seeds. However, from game perspective this doesn't affect `random` function specification/usage.

Simulation Engine

Simulation engine makes it fast & easy to run simulations of your game server.

```
npx slotify-gdk stats-multiplier -g games/my_game -i 5m
```

Usage: `slotify-multiplier-stats [options]`

Options:

<code>-g, --game <string></code>	path to the game (required)
<code>-i, --iterations <number></code>	number of iterations to simulate (default: <code>1000000</code>)
<code>-r, --rngAlgorithm <string></code>	rng algorithm (isaac, mersenne-twister) (default: <code>"isaac"</code>)
<code>-c, --cores <number></code>	number of cores (default: <code>"8"</code>)
<code>-s, --strategy <string></code>	simulation strategy
<code>-l, --lag <string></code>	command lag probability (in percentage)
<code>-h, --help</code>	display help for command

How to use system commands

If you want to use system commands, you will need to connect directly to the WebSocket. To do it you will need to obtain Game Room secret key.

Following function is used to create signature for **HMAC** authentication:

```
async function createJsonHmacSignature(message: string, secretKey) {  
  const encoder = new TextEncoder();  
  const algorithm = { name: "HMAC", hash: { name: "SHA-256" } };  
  
  const key = await crypto.subtle.importKey(  
    "raw",  
    encoder.encode(secretKey),  
    algorithm,  
    true,  
    ["sign"]  
  );  
  const signature = await crypto.subtle.sign(algorithm, key, encoder.encode(message));  
  return signature;  
}
```

```

    "raw",
    encoder.encode(secretKey),
    algorithm,
    false,
    ["sign"]
  );

  const signatureBuffer = await crypto.subtle.sign("HMAC", key, encoder.encode(message));

  const hexSignature = Array.from(new Uint8Array(signatureBuffer))
    .map(b => b.toString(16).padStart(2, '0'))
    .join('');

  return hexSignature;
}

```

Following example shows how to open WebSocket connection:

```

const secretKey = "my-secret-key";
const channel = "12d42bae-9684-40bd-a38d-d764259bb053";
const systemId = "6923b28d-fcd5-4a32-bfef-9bd891aed34d";

const connectedSignature = await createJsonHmacSignature(JSON.stringify({channel, systemId}), secretKey);
const wsParams = `channel=${channel}&type=system&systemId=${systemId}&signature=${connectedSignature}`;
const websocket = await new WebSocket(`ws://localhost:8087/websocket/multiplayer?${wsParams}`);

```

- `channel` - room id
- `secretKey` - room secret key
- `systemId` - UUID v4 id (mandatory to be UUID v4)

Following example shows how to send system commands through WebSocket:

```

const load = {
  type: 'systemCommand',
  payload: {
    action: 'cardReveal',
    params: {
      //your params here
    },
  },
};

const message = JSON.stringify(load);
const messageSignature = await createJsonHmacSignature(
  message,
  secretKey,
);
const sendMessage = `${messageSignature}\n--SIG--\n${message}`;

ws.send(sendMessage);

```

`sendMessage` should exactly follow this structure `<messageSignature>\n--SIG--\n<message>`